Scaling AI workloads on HPE Cray EX

Jessica Jones
Principal Research Engineer, HPC/AI EMEA Research Lab

Tuesday 2nd September 2025

Agenda

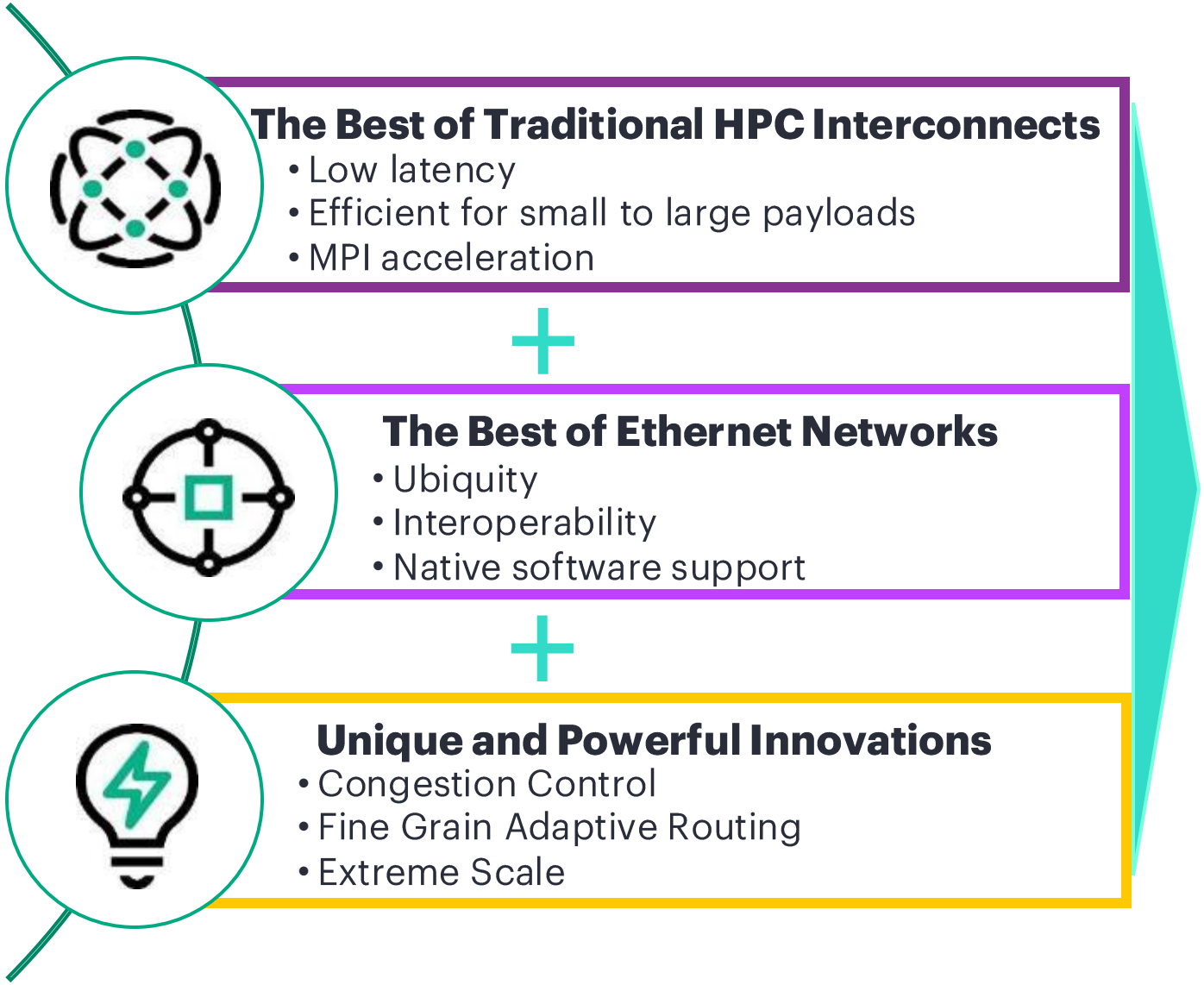
Scaling AI workloads beyond a single node

01	Slingshot
02	PyTorch Distributed
03	DistributedDataParallel
04	Fully Sharded Data Parallel 2 (FSDP2)
05	Pipelines
06	Questions?



Slingshot

SLINGSHOT



HPE Slingshot

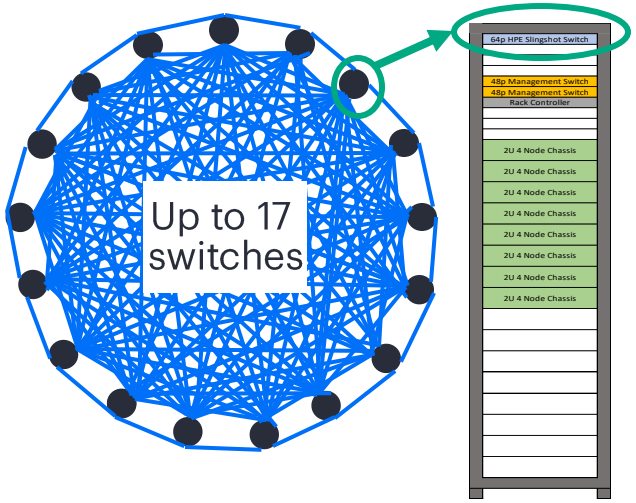
- ✓ Performance – low latency & high BW – realized consistently under load
- ✓ Native standard Ethernet - connectivity to data center resources and data sources
- ✓ Optimized and supported as a solution from silicon to systems to software
- ✓ Ideal for systems that mix tightly-coupled HPC, AI, and Analytics workloads

© 2025 Hewlett Packard Enterprise Development LP

HPE Slingshot Topologies

Full Mesh

Up to 32 endpoints per switch

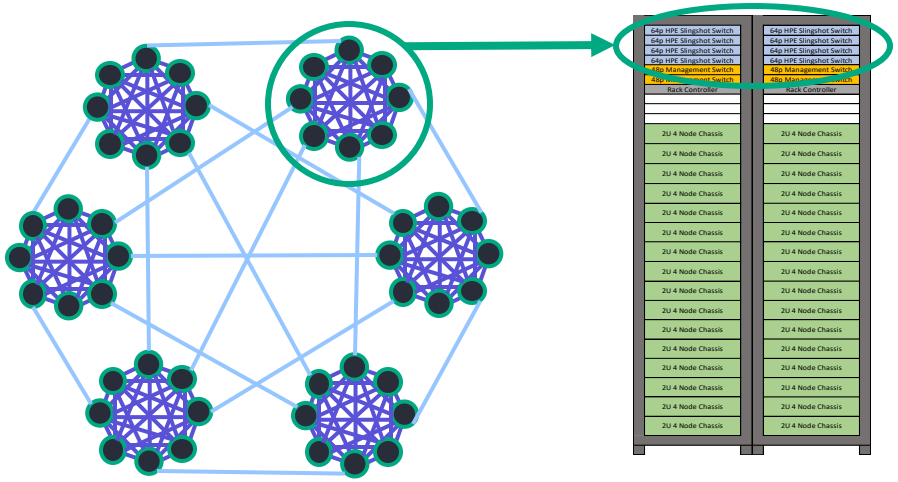


- Up to 17 fully connected switches
- Each 32 NICs up to 544 total
- Very cost-effective for clusters
- One hop minimal path

Dragonfly

Up to 16 endpoints per switch

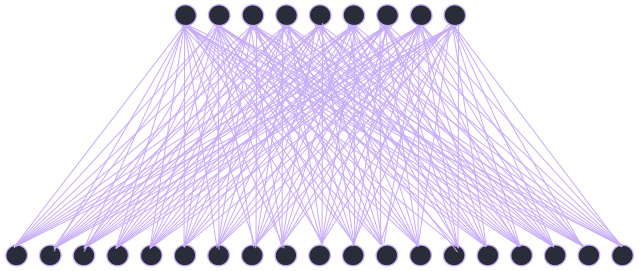
Illustrated: 8 switch group ➔ 128 endpoints



- All-to-all group connects to every group
- Maximize copper for lower cost, less power
- Group size optimizes for cost, scale, deployment
- <= 3 switch-to-switch hops up to 250K endpoints
- Best performance for larger systems

Fat Tree

Targeting H2 2026 Completion



- Easier to continually add capacity
- Cost advantage possible between 544 and 1024 endpoints and lower-performance use cases

© 2025 Hewlett Packard Enterprise Development LP

HPE Cray Supercomputing EX2500 – Slingshot Fabric

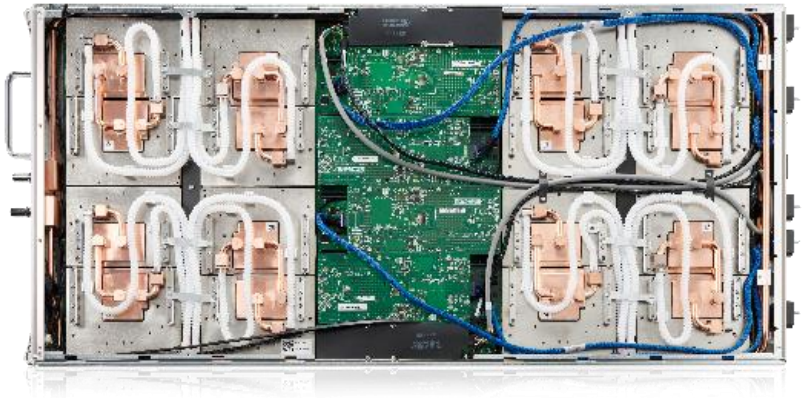
HPE Slingshot Switch (Direct Liquid Cooled)

- 64 ports @ 200 Gbps Per Port (in 32 DD jacks)
- Up to four (4) Slingshots switches per chassis (12 switches per cabinet)



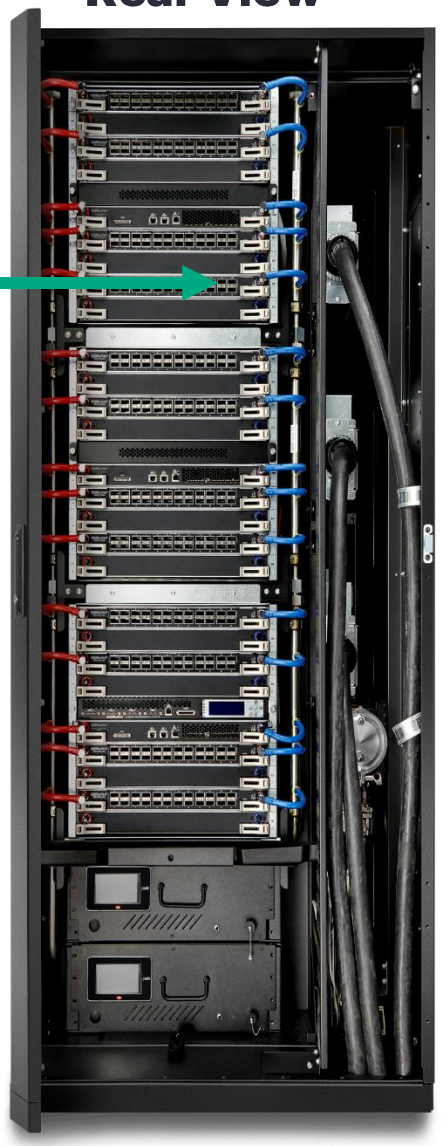
NIC Mezzanine Slingshot 11 (200 Gbps)

- HPE Cray Supercomputing EX255a / EX254n GPU Accelerated Blades (4 injection ports per node)
- HPE Cray Supercomputing EX425 /EX4252 Compute Blades (1 or 2 Injection ports per node)

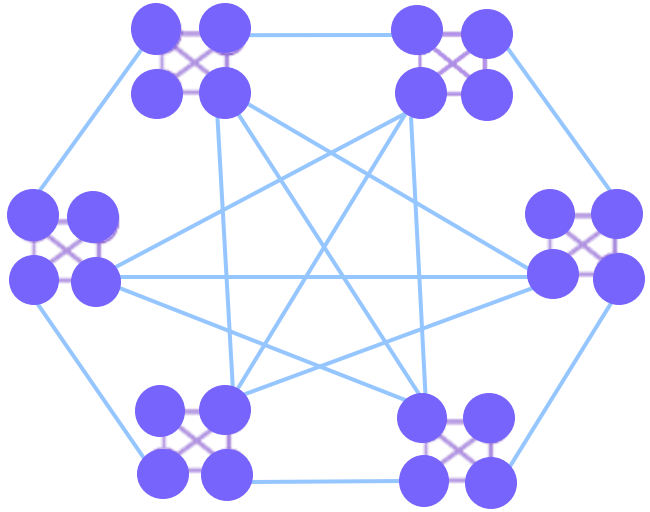


HPE Cray EX254n Compute Blade

Rear View



2 Rack / 4 Switch Group Dragonfly topology



- Each chassis is its own Dragonfly Group
- Up to (1) 200Gbs injection port per socket
- How to determine your switch group
  - HPE Cray Supercomputing EX425 /EX4252 Compute Blades
    - 1 Injection port per node = 2 switch group
    - 1 Injection port per socket= 4 switch group
  - HPE Cray Supercomputing EX255a / EX254n GPU Accelerated Blades
    - 4 injection ports per node GPU blades= 4 switch group

© 2025 Hewlett Packard Enterprise Development LP

HPE SLINGSHOT – HOW IT WORKS

Ethernet links to NICs, Edge Switches

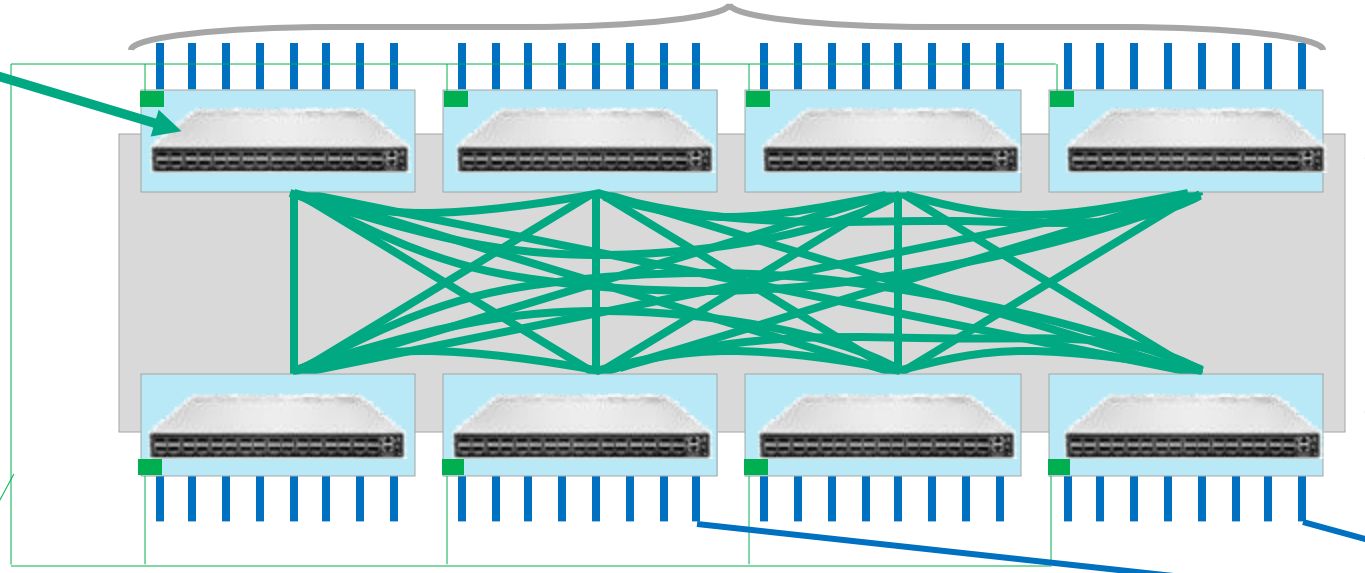
Ethernet physicals, protocols and flow control

(Enhanced packet efficiency, edge links resilience and fine-grained flow control with HPE Slingshot NICs)

HPE Slingshot Switches

- ToR or Blade

Switch ports configured as "edge" or "fabric"



Fabric links connect switches

- Enhanced link resilience
- Networking innovations tackle congestion on links & endpoints
  - Fine Grained Adaptive Routing
  - Credit-Based Flow Control
  - Congestion Management

Out-of-Band Mgt Network

e.g. Aruba 8325

Wide MLAG from any switch





Fabric management

- Configuration
- "Routing" (forwarding) updates
- Monitoring/Health

Dedicated or containerized Linux OS

Edge Switches (Optional)

- Intranet & internet connections
- Ethernet resources
- Aruba and qualified 3rd party

© 2025 Hewlett Packard Enterprise Development LP

Cableless Network Connection to Liquid Cooled HPE Slingshot Fabric Switches

QSFP-DD Cables

Group cables - PEC

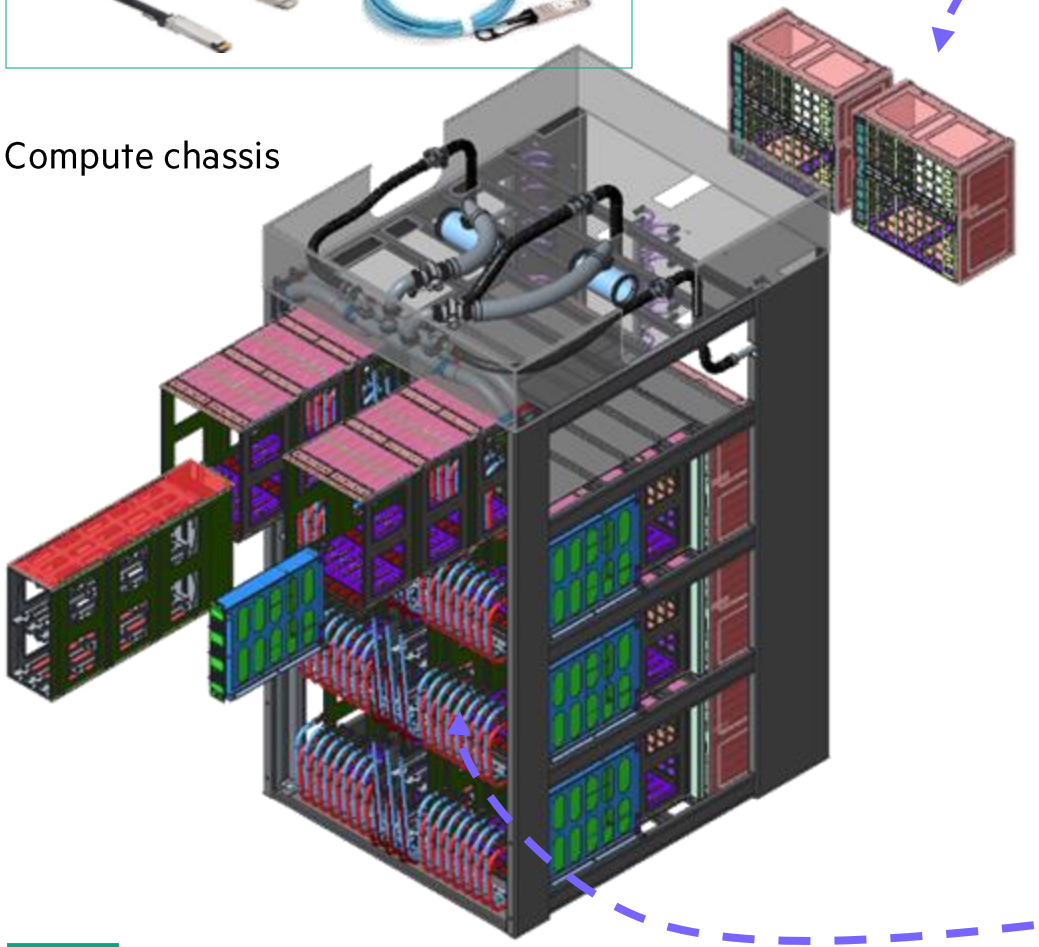


Global cables - AOC



Switch Chassis

Compute chassis



HPE Slingshot Switch (Direct Liquid Cooled)

- 64 ports @ 200 Gbps Per Port (in 32 DD jacks)
- Up to four (4) Slingshots switches per chassis (32 switches per cabinet)

Rear - to fabric



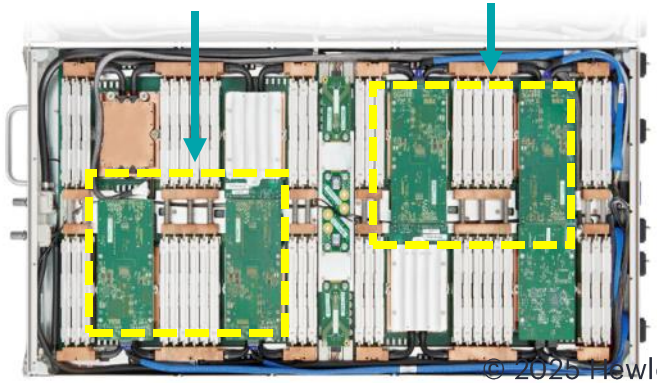
Group/Global links

Internal – mate to blade



NIC Mezzanine Slingshot 11 (200 Gbps)

- HPE Cray Supercomputing EX425 / EX4252 Compute Blades (1 or 2 Injection ports per node)
- HPE Cray Supercomputing EX255a / EX254n GPU Accelerated Blades (4 injection ports per node)

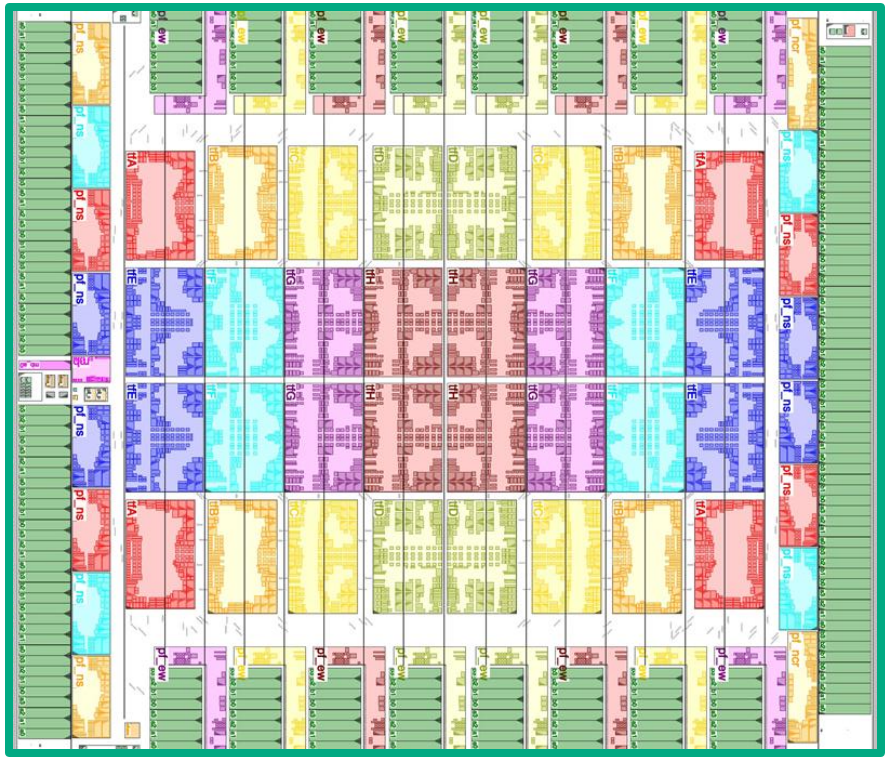


HPE Cray EX4252 Compute Blade

Switch and NICs mate mid-cabinet at 90˚ orientation

© 2025 Hewlett Packard Enterprise Development LP

Switch components





© 2025 Hewlett Packard Enterprise Development LP

Networking at Exascale



Lawrence Livermore National Laboratory



OAK RIDGE National Laboratory



Argonne NATIONAL LABORATORY



CSCS



NERSC



pawsey



EuroHPC Joint Undertaking

CSC



NREL NATIONAL RENEWABLE ENERGY LABORATORY



Los Alamos NATIONAL LABORATORY



KAUST



US Air Force Weather (ORNL)



INES

June '25 Top500 w/HPE Slingshot

#1	LLNL El Capitan
#2	ORNL Frontier
#3	ANL Aurora
#6	ENI HPC6
#8	CSCS Alps
#9	CSC LUMI
#11	Bristol Isambard-AI phase 2
#12	LLNL Tuolumne
#17	Exxon Discovery 6
#19	LANL Venado
#25	NERSC Perlmutter
#28	Sandia El Dorado
#40	GENCI-CINES Adastra
#47	KAUST Shaheen III - CPU
#54	HLRS Hunter
#57	LANL Crossroads
#59	PSC Setonix - GPU
#60	Exxon Discovery 5
#61	ANL Polaris
#64	LLNL rzAdams
#65	HPE Portage
#79	UE Archer2
#83	Saudi Aramco Ghawar-1
#84	ORNL Frontier TDS
#85	Cyfronet Helios GPU
#87	NREL Kestrel GPU
#89	NCSA DeltaAI
#117	Sigma2 Olivia (GPU)
#126	ERDC DSRC Carpenter

Plus 30 more!

Top500.org, June 2025 list

© 2025 Hewlett Packard Enterprise Development LP

Networking at Exascale

Frontier Slingshot Fabric

Compute nodes

- 9408 compute nodes each with 4 Cassini NICs
- Peak network bandwidth of 800Gbps/dir per node

Dragonfly network: 74 compute groups of 32 Rosetta switches

- 2,464 Rosetta switches
- 37,632 Cassini NICs (18,816 ASICs)
- 18,816 L0 cable assemblies (2 links each)
- 9,472 L1 cable assemblies (4 links each)
- 5,402 L2 active optical cables (2 links each).
- 161,844 active 200Gbps fabric ports
- 5 storage groups, each 292 ports



Aurora Slingshot Fabric

Compute nodes

- 10624 compute nodes each with 8 Cassini NICs
- Peak network bandwidth of 1600Gbps/dir per node

Dragonfly network: 166 compute groups of 32 Rosetta switches

- 5,600 Rosetta switches
- 87,404 Cassini NICs (43,702 ASICs)
- 43,702 L0 cable assemblies (2 links each)
- 21,851 L1 cable assemblies (4 links each)
- 15,557 L2 active optical cables (2 links each)
- 241,428 active 200Gbps fabric ports
- 9 DAOS I/O groups



© 2025 Hewlett Packard Enterprise Development LP

Isambard-AI Network Overview

Isambard Slingshot Fabric

Compute nodes

— 5280 compute nodes each with 4 Cassini NICs, peak bandwidth of 800Gbps/dir

— Dragonfly network 42 groups of 20 switches

— 128 nodes per group

— 21120 Cassini NICs



Network properties

— Injection bandwidth of 800 Gbps per node

— Local link bandwidth is 200% of injection

— Global link bandwidth is 57% of injection

Slingshot

Ethernet (IEEE 802.3cd) links with Slingshot specific additions for performance and reliability

— Congestion management

— Adaptive routing

— High rate of small packets

— Link level retry

HPE/Cray programming environment

— HPE/Cray MPI

— Libfabric provider for Cassini NIC

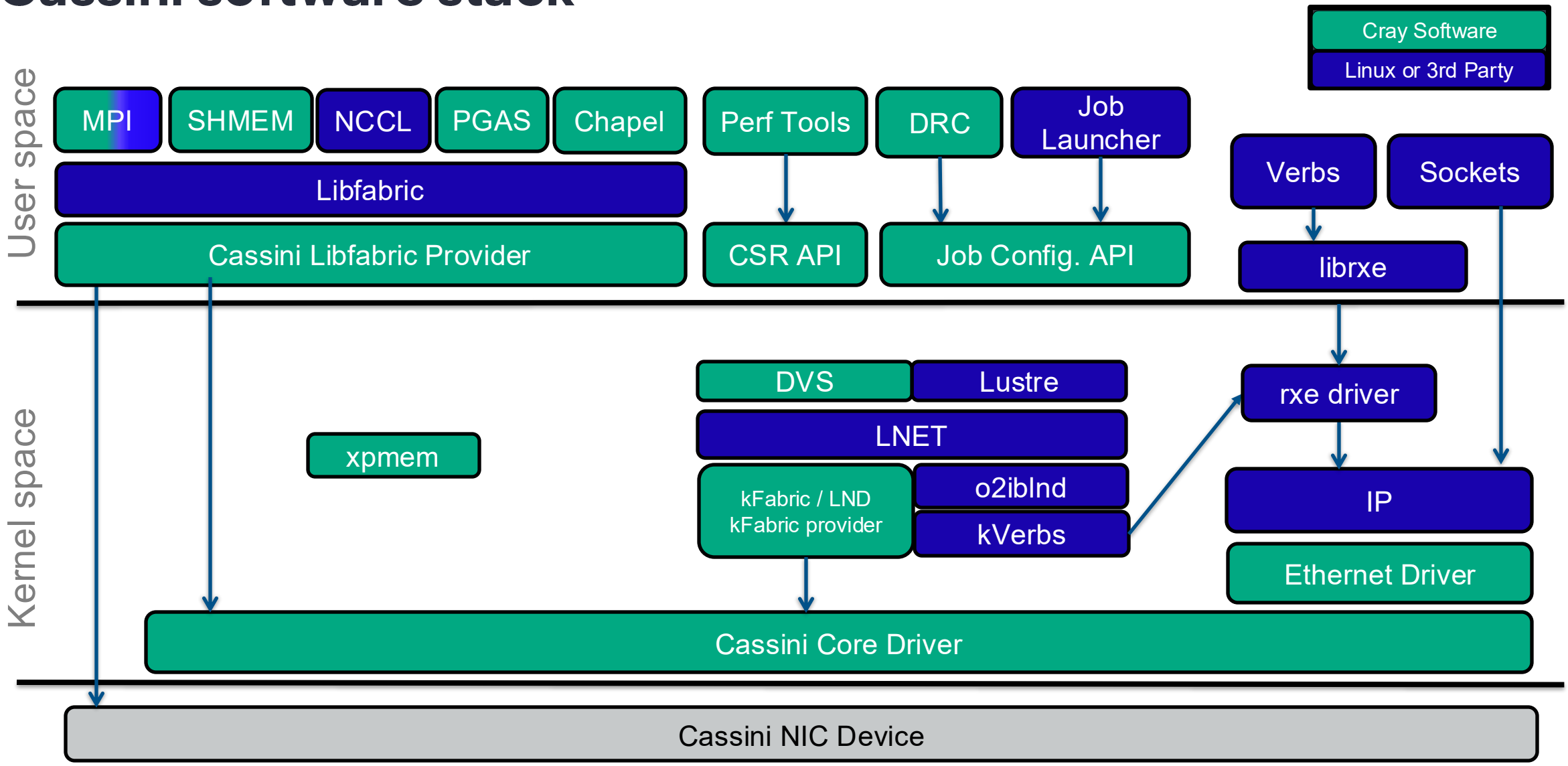
Embedded management and monitoring system

— 840 management agents (one per switch)

— Management Ethernet network 100Gbps per switch

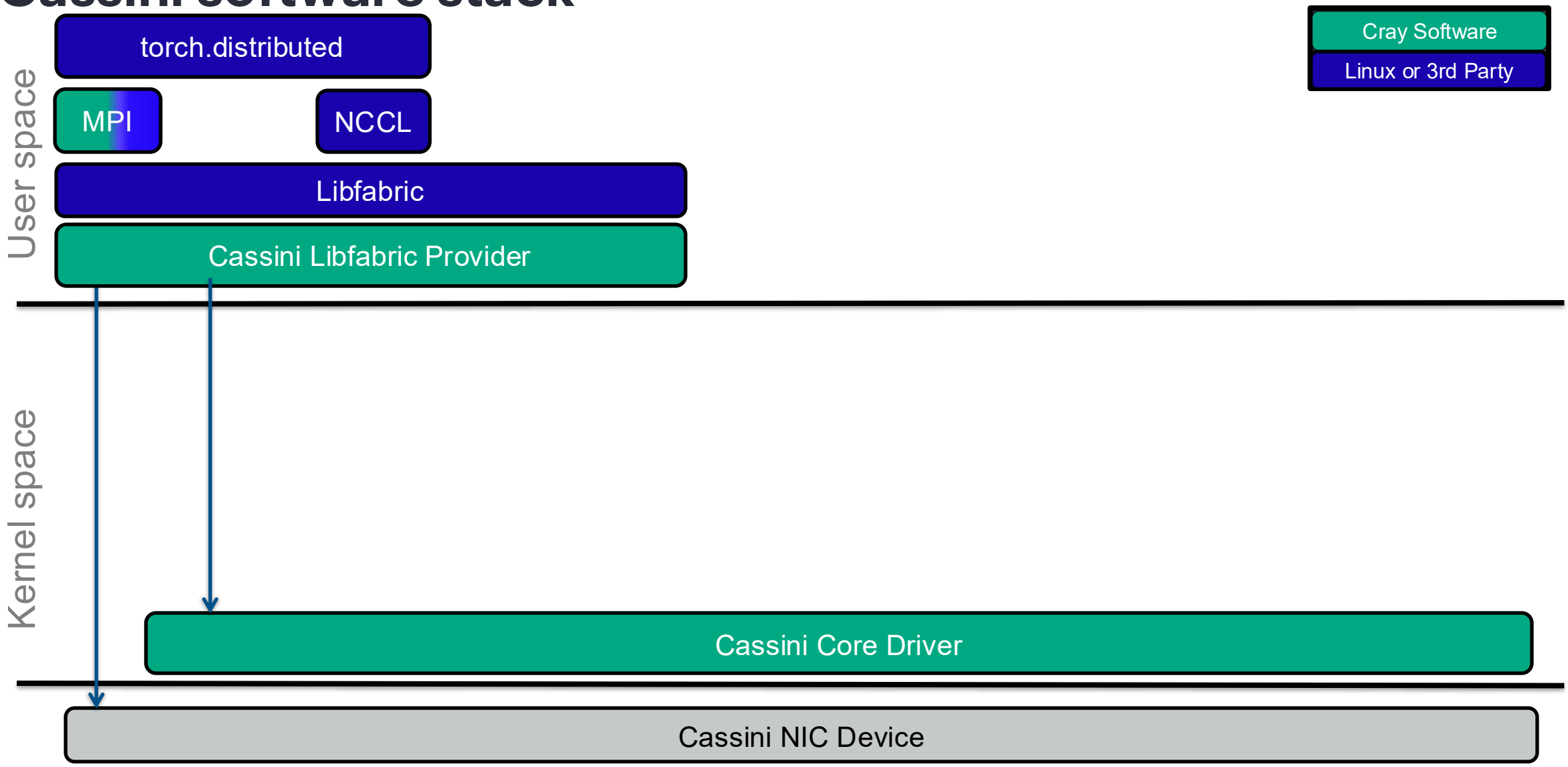
© 2025 Hewlett Packard Enterprise Development LP

Cassini software stack



© 2025 Hewlett Packard Enterprise Development LP

Cassini software stack



© 2025 Hewlett Packard Enterprise Development LP

PyTorch Distributed



© 2025 Hewlett Packard Enterprise Development LP

PyTorch Distributed

- PyTorch has a built-in module for distributed communication
  - torch.distributed
  - https://docs.pytorch.org/docs/stable/distributed.html

- Point to point and collective communication methods.
- Sharded primitives via DTensor
- DeviceMesh provides a way to tune collective communication

- Includes several useful parallelism modules.
  - DataParallel -- now deprecated
  - DistributedDataParallel (DDP)
  - Fully Sharded Data-Parallel Training 2 (FSDP2)
  - Tensor Parallel (TP)
  - Pipeline Parallel (PP)

- To use these, your PyTorch installation *must* be built with USE_DISTRIBUTED=1
  - Use `torch.distributed.is_available()` to check for support



© 2025 Hewlett Packard Enterprise Development LP

PyTorch Distributed

- PyTorch has a built-in module for distributed communication
  - torch.distributed
  - https://docs.pytorch.org/docs/stable/distributed.html

- Point to point and collective communication methods.
- Sharded primitives via DTensor
- DeviceMesh provides a way to tune collective communication

- Includes several useful parallelism modules.
  - ~~DataParallel - now deprecated~~
  - **DistributedDataParallel (DDP)**
  - Fully Sharded Data-Parallel Training 2 (FSDP2)
  - Tensor Parallel (TP)
  - Pipeline Parallel (PP)

- To use these, your PyTorch installation *must* be built with USE_DISTRIBUTED=1
  - Use `torch.distributed.is_available()` to check for support



© 2025 Hewlett Packard Enterprise Development LP

Network backends

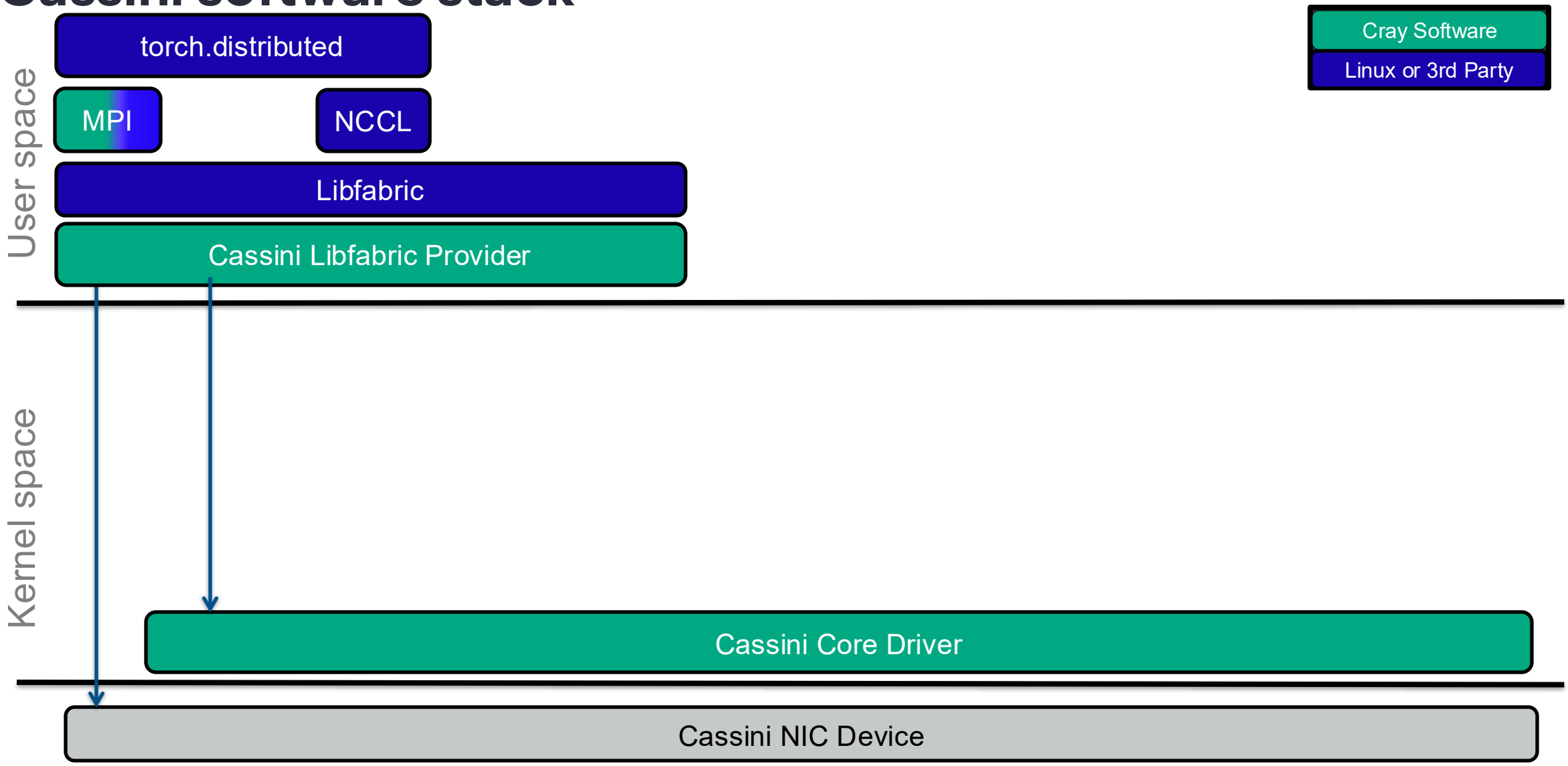
- `torch.distributed` supports several backends via the c10d library (implemented in C++):
  - Gloo
  - NCCL
  - MPI

- *There are also others*



© 2025 Hewlett Packard Enterprise Development LP

Cassini software stack



© 2025 Hewlett Packard Enterprise Development LP

Network backends

- `torch.distributed` supports several backends via the c10d library (implemented in C++):
  - Gloo
  - NCCL
  - MPI

- For distributed applications, you will can either launch with a combination of Slurm's srun and torchrun *simultaneously,* ***or*** *just use srun*

- If you install from pip in the usual way, you will have gloo and NCCL available, but *not MPI*

```
python -m pip install torch torchvision torchaudio \
                --index-url https://download.pytorch.org/whl/cu128
```



© 2025 Hewlett Packard Enterprise Development LP

Network backends

```
if torch.distributed.is_available():
      if torch.distributed.is_nccl_available():
              backend=“nccl”
      elif torch.distributed.is_mpi_available():
              backend=“mpi”
      else:
              backend=“gloo”

      torch.distributed.init_process_group(backend=backend, …)
```



© 2025 Hewlett Packard Enterprise Development LP

Setting up distributed PyTorch communication

- You can select the distributed communication backend
  - For multi-GPU we use the NCCL backend
- You can initialize the torch.distributed process group in multiple ways
  - `torch.distributed.init_process_group()`
  - TCP, Shared Filesystem, Environment
- Our example will use the *environment* approach

Initialisation:

```
dist.init_process_group(backend='nccl',
                        init_method='env://',
                        world_size=int(os.environ['WORLD_SIZE']),
                        rank=int(os.environ['RANK']))
```

WORLD_SIZE: How many processes are there?
RANK: Which one am I?

Convenience script:

```
#!/bin/bash

# Set environment for each MNIST process
export MASTER_ADDR=$(python3 ./get_master.py "$SLURM_NODELIST")
export MASTER_PORT=29500
export WORLD_SIZE=$SLURM_NPROCS
export RANK=$SLURM_PROCID

# Run the example
python3 -u mnist_DDP.py --gpu --modelpath model
```

MASTER_ADDR: Hostname of master process
MASTER_PORT: Free port for communication



© 2025 Hewlett Packard Enterprise Development LP

Example: srun and torchrun combined

```
#SBATCH --nodes=2
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=8
#SBATCH --gpus-per-task=4

export OMP_NUM_THREADS=6
nodes=( $( scontrol show hostnames $SLURM_JOB_NODELIST ) )
nodes_array=($nodes)
head_node=${nodes_array[0]}
head_node_ip=$(srun --nodes=1 --ntasks=1 -w "$head_node" hostname --ip-address)

cd $SLURM_SUBMIT_DIR
echo "Node IP: ${head_node_ip}”
. .venv/dpp/bin/activate


srun --ntasks-per-node 1 -N 2 \
    torchrun \
                --nnodes ${SLURM_NNODES} \
                --nproc_per_node 4 \
                --rdzv_id $RANDOM \
                --rdzv_backend c10d \
                --rdzv_endpoint $head_node_ip:29500 \
                ${SLURM_SUBMIT_DIR}/demo.py --total_epochs 100 --save_every 30 --batch_size 64 \
                --data_dir "${SLURM_SUBMIT_DIR}/data/"
```



© 2025 Hewlett Packard Enterprise Development LP

Example: srun and torchrun combined

```
#SBATCH --nodes=2
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=8
#SBATCH --gpus-per-task=4

export OMP_NUM_THREADS=6
nodes=( $( scontrol show hostnames $SLURM_JOB_NODELIST ) )
nodes_array=($nodes)
head_node=${nodes_array[0]}
head_node_ip=$(srun --nodes=1 --ntasks=1 -w "$head_node" hostname --ip-address)

cd $SLURM_SUBMIT_DIR
echo "Node IP: ${head_node_ip}"
. .venv/dpp/bin/activate


srun --ntasks-per-node 1 -N 2 \
    torchrun \
                --nnodes ${SLURM_NNODES} \
                --nproc_per_node 4 \
                --rdzv_id $RANDOM \
                --rdzv_backend c10d \
                --rdzv_endpoint $head_node_ip:29500 \
                ${SLURM_SUBMIT_DIR}/demo.py --total_epochs 100 --save_every 30 --batch_size 64 \
                --data_dir "${SLURM_SUBMIT_DIR}/data/"
```

torchrun provides process_group initialisation details through environment variables:

WORLD_SIZE, RANK, LOCAL_RANK

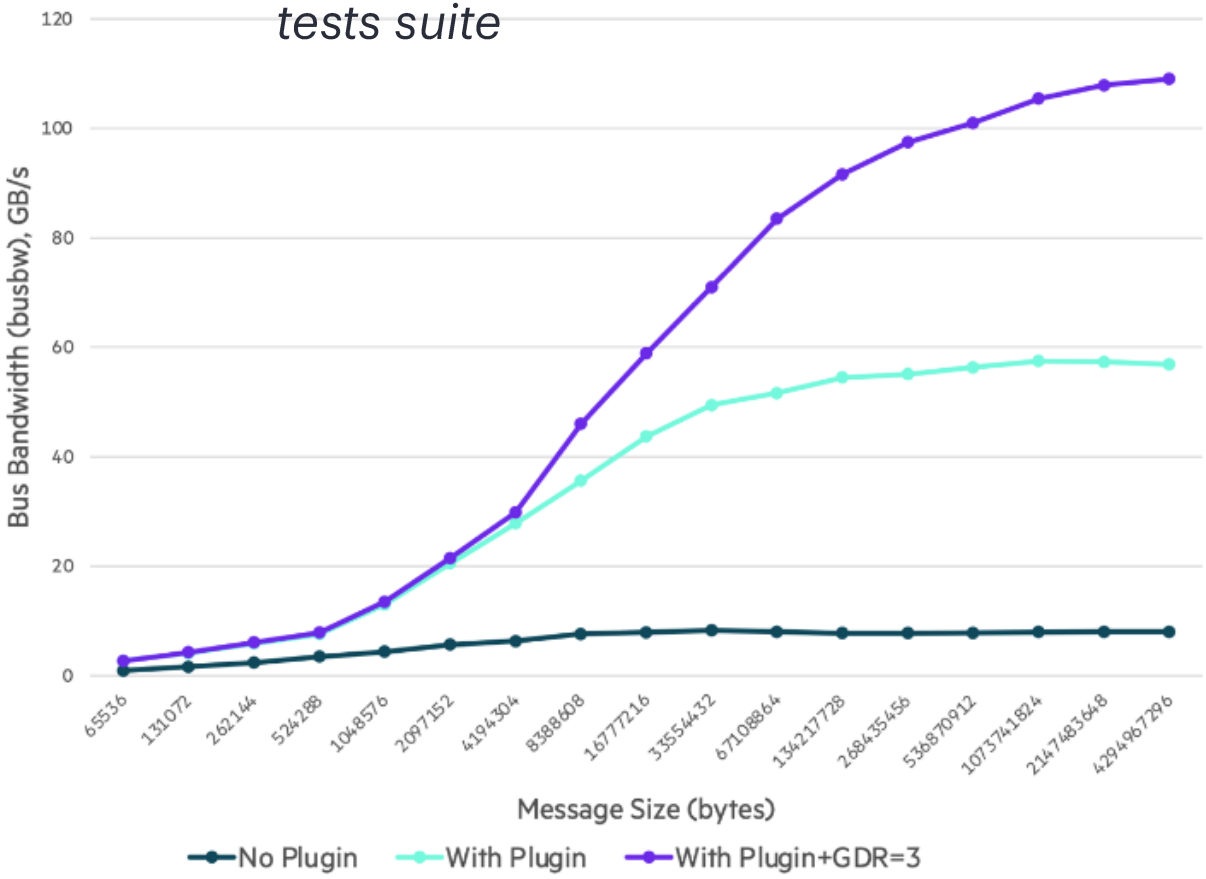


© 2025 Hewlett Packard Enterprise Development LP

GPU communications for multi-node pytorch

- Communication is often a bottleneck for multi-node and multi-GPU applications
- PyTorch distributed backend for GPUs uses NCCL/RCCL as underlying communication library
  - RCCL: ROCm Communication Collectives Library
- Default is TCP/IP – not getting the performance boost of Slingshot or GPU RDMA!
  - To get that performance boost, we need some additional configuration
  - Also requires the AWS-OFI-NCCL plugin to allow RCCL to benefit from libfabric optimized transport layer
- Properly using the interconnect requires some configuration, but provides good performance increase
  - 3-5x better bandwidth utilization with plugin
  - 2x better bandwidth utilization with RDMA

RCCL Bandwidth Tests via rccl-tests suite



© 2025 Hewlett Packard Enterprise Development LP

Tuning NCCL performance on Slingshot

- NCCL interfaces to Slingshot via libfabric via the aws-ofi-nccl plugin
- Tuning of libfabric and NCCL variables is possible
- On most EX systems, tuning should have been done for you
  - This is good because there are hundreds of variables!

- Sometimes there are still things you could do to improve things for specific applications

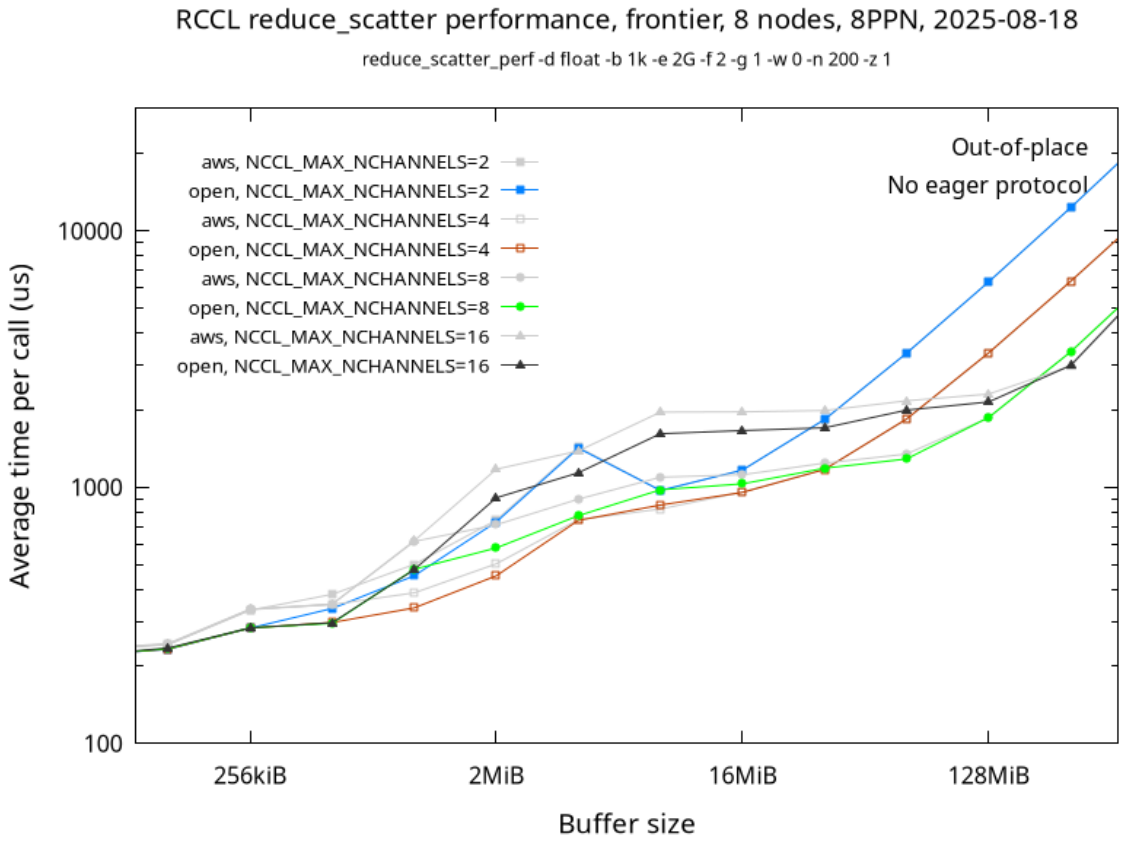


© 2025 Hewlett Packard Enterprise Development LP

Tuning NCCL performance on Slingshot

- NCCL schedules point to point I/O via the aws-ofi-nccl plugin, then via libfabric
- Adjusting the algorithm that NCCL uses can *sometimes* improve performance at larger job sizes
- Compare NCCL_ALGO=RING with NCCL_ALGO=TREE
- Adjust the number of channels that NCCL uses:
  - NCCL_MAX_CHANNELS

A simple sweep with each algorithm can sometimes pay off with improved performance.



© 2025 Hewlett Packard Enterprise Development LP

Process pinning

- Isambard-AI has 4 NICs per node
- It is important that processes *do not move* between NICs or NUMA regions

- You can use the usual affinity tools to pin processes
  - taskset
  - numactl
  - Slurm pinning options
    - `--cpu-bind` is probably the easiest, but too simplistic unless you use `map`

- NCCL requires a core of its own
- So does the OS



© 2025 Hewlett Packard Enterprise Development LP

numactl

You could choose to use a snippet like this in your Slurm job script:

```
NUMA_CMD=""
if [ $SLURM_LOCALID -eq 0 ] ; then
  NUMA_CMD=" numactl -a --membind=0-3 --physcpubind=1-43,45-70 "
elif [ $SLURM_LOCALID -eq 1 ] ; then
  NUMA_CMD=" numactl -a --membind=0-3 --physcpubind=73-110,112-142 "
elif [ $SLURM_LOCALID -eq 2 ] ; then
  NUMA_CMD=" numactl -a --membind=0-3 --physcpubind=145-182,184-214 "
else
  NUMA_CMD=" numactl -a --membind=0-3 --physcpubind=217-255,257-286 "
fi

 ${NUMA_CMD} python example.py
```

This prevents your application using the 1st core in each NUMA region so that it can be used by NCCL

© 2025 Hewlett Packard Enterprise Development LP

Debugging Collective Performance

Some environment variables may still be of interest if debugging:

- *IFF* using NCCL:
  - `export NCCL_DEBUG=INFO` *(for everything – lots of output)*
  - `export NCCL_DEBUG_SUBSYS=COLL` *(for collectives only)*
- *IFF* using MPI, this may be helpful in debugging:
  - `export MPICH_OFI_CXI_COUNTER_REPORT=2`
- If you want to compare, you can use FI_PROVIDER to force libfabric to use TCP/IP
  - `export FI_PROVIDER=tcp`



© 2025 Hewlett Packard Enterprise Development LP

DistributedDataParallel



© 2025 Hewlett Packard Enterprise Development LP

What is DistributedDataParallel (DDP)?

- Normal training:
  - Model receives input batch from DataLoader
  - Forward pass to calculate loss
  - Backward pass to calculate perimeter gradients
  - Perimeter gradients are used by the Optimiser to update the model



© 2025 Hewlett Packard Enterprise Development LP

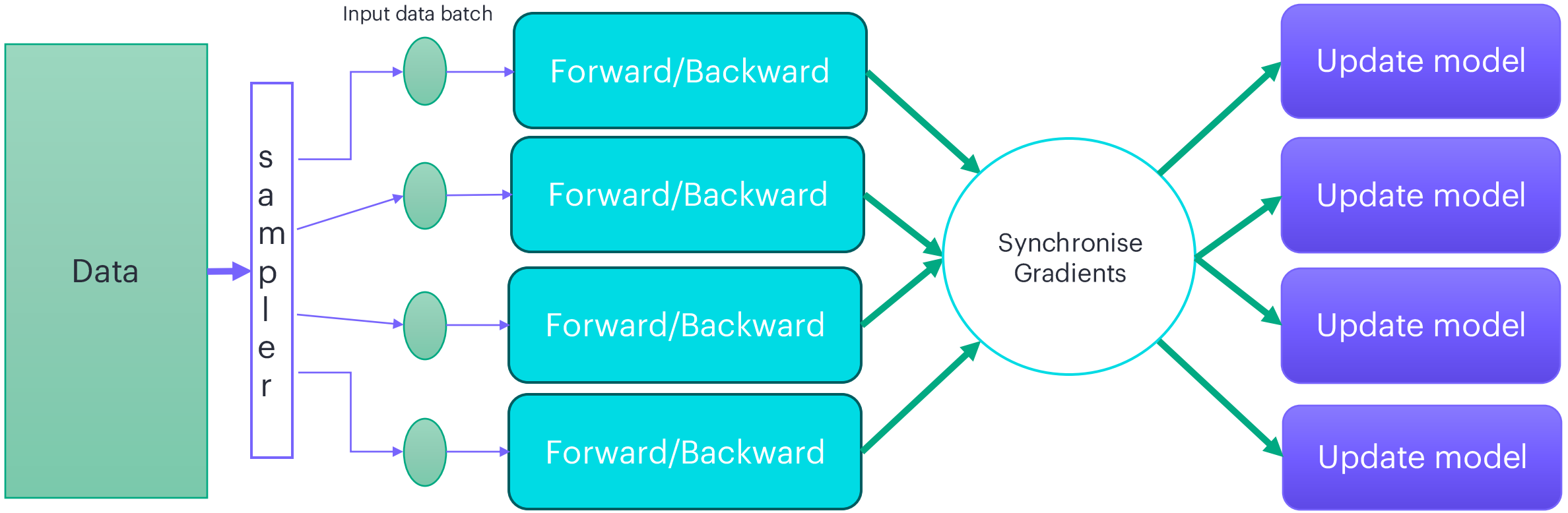
What is DistributedDataParallel (DDP)?

- DDP training:
  - Starts one process per device
    - Device can be CPU, GPU or something else
    - Each process has *its own local copy* of the model
    - Model, initial parameters, optimisers, etc are all identical
    - PRNGs are seeded identically
  - Model *on each process* receives input batch from DataLoader, as before
    - Uses DistributedSampler; correctly distributed input batch
    - *Only the data each process receives is different*
  - Forward pass to calculate loss
  - Backward pass to calculate perimeter gradients – *These are now different on each process!*
  - SYNCHRONISE gradients using bucketed all_reduce()
  - Perimeter gradients are used by the Optimiser to update the model



© 2025 Hewlett Packard Enterprise Development LP

DistributedDataParallel



© 2025 Hewlett Packard Enterprise Development LP

DistributedDataParallel

- Provides data parallelism by *synchronizing* gradients across each model replica
- You simply wrap the standard PyTorch model
- The `process_group` specifies the devices to synchronise across
- Each process has a copy of the optimiser and works on its own copy of the data
  - Reduce I/O at cost of computation

- You are responsible for deciding how to split up data across devices
  - `DistributedDataSampler` is provided for convenience



© 2025 Hewlett Packard Enterprise Development LP

Using DistributedDataParallel

- Import required modules
  - `From torch.utils.data.distributed import DistributedSampler`
  - `From torch.nn.parallel import DistributedDataparallel as DDP`
  - `From torch.distributed import init_process_group, destroy_process_group`
  - `Import os`
- Obtain `world_size`, `rank` and `local_rank` from `os.environ`
- Add call to `init_process_group` and set the target device ID
- Pass a `DistributedSampler` to the `DataLoader`
- Wrap the model in `DistributedDataParallel`
- Check that all references to the model are compatible with the DDP object
- Add a call to `destroy_process_group` to ensure that we exit cleanly after training is complete

- *We will walk through this together during the practical*



© 2025 Hewlett Packard Enterprise Development LP

FSDP2

Fully-Sharded Data Parallel 2



© 2025 Hewlett Packard Enterprise Development LP

Fully-Sharded Data Parallel 2

- In DDP, we saw that each process has an identical copy of the model, with its own data
  - There is a lot of duplication and models must still fit into a single GPU
- FSDP2 reduces memory usage by sharding model gradients, parameters and optimizer states
  - Frees you from having to ensure that the model fits within a single GPU

- Parameters are fully sharded outside of forward and backward computation
- Before forward and backward passes, sharded parameters are all-gathered into unsharded parameters
- Inside backward, local unsharded gradients are reduce-scattered into sharded gradients
- Optimizer then updates sharded parameters with sharded gradients, resulting in sharded optimizer states

- Replaces each all-reduce call in DDP with several reduce-scatter and all-gather calls



© 2025 Hewlett Packard Enterprise Development LP

Using FSDP2

- Similar to DDP: Wrap your model, then initializing your optimizer
  - Our nn.Linear in the CNN becomes a union of nn.Linear and FSDPModule; FSDPLinear
  - Sharding your model must be done “bottom up” so that each layer is sharded first
- Must be done in that order as the parameters are changed by FSDP2
- Parameters are now DTensor instances and are fully sharded
- Pre-forward/backward hooks do all-gather of parameters and conversion from DTensor back to torch.Tensor
- Post-forward/backward hooks reverse this and convert the unsharded Tensors to DTensors



© 2025 Hewlett Packard Enterprise Development LP

Using FSDP2

```
from torch.distributed.fsdp import fully_shard, FSDPModule
..
model = Transformer()
for layer in model.layers:
        fully_shard(layer)
fully_shard(model)
..
```

Then initialise your optimiser.



© 2025 Hewlett Packard Enterprise Development LP

Pipelines



© 2025 Hewlett Packard Enterprise Development LP

Distributed Pipeline Parallism

- `torch.distributed.pipelining` provides tools to to partition the model execution and schedule computation in micro-batches
- This can be useful for specific types of training workloads, such as transformer-decoder models
- Requires more code changes than DDP or FSDP2
- Still uses torchrun as a helper script to launch



© 2025 Hewlett Packard Enterprise Development LP

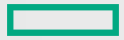
Questions?



© 2025 Hewlett Packard Enterprise Development LP

Practical Session

Parallelising our MNIST example with DDT



© 2025 Hewlett Packard Enterprise Development LP

Initialising our process group: Step 1

Harvest process group initialization data from our local environment

```
import os
```

(need an import at the start of program to access os module)

```
# Pull our global ‘rank’ id and the ‘world size’ from our environment
if 'RANK' in os.environ and 'WORLD_SIZE' in os.environ:
  rank = int(os.environ['RANK'])
  world_size = int(os.environ['WORLD_SIZE'])
else:
  rank = 0
  world_size = 1

# Pull our local rank id, used to index GPUs with a node when we have multiple per node as in Isambard AI
if 'LOCAL_RANK' in os.environ:
   local_rank = int(os.environ['LOCAL_RANK'])
else:
   ngpus = torch.cuda.device_count() if use_cuda else 1
   local_rank = None
```

(we will populate the environment with this data later!)



© 2025 Hewlett Packard Enterprise Development LP

Initialising our process group: Step 2

Use data from Step 1 to initialize a process group via torch.distributed

```
import torch.distributed as dist
```

(need an import at the start of program to access distributed module)

```
# Init a process group via environment, use NCCL backend if gpus are available
try:
  if world_size > 1:
    dist.init_process_group(
      backend='nccl' if use_cuda and dist.is_nccl_available() else 'gloo',
      init_method='env://',
      rank=rank,
      world_size=world_size
    )
except Exception as e:
  print(f"Error initializing process group: {e}")
  exit(1)
```



© 2025 Hewlett Packard Enterprise Development LP

Select the appropriate GPU

We have multiple nodes, each with multiple GPUs, so we need to ensure each of our processes selects the correct GPU

```
# If we are running on gpu, use local rank to choose device within a node
if use_cuda:
  if local_rank >= torch.cuda.device_count():
    print(f"Requested CUDA device {local_rank} not available.")
    exit(1)
  device = torch.device(f"cuda:{local_rank}")
  torch.cuda.set_device(local_rank)
elif use_mps:
  device = torch.device("mps")
else:
  device = torch.device("cpu")
```



© 2025 Hewlett Packard Enterprise Development LP

Clean up our process group at program end

If we don’t clean up our process group, resources might be leaked

In general, SLURM or the OS will kill them anyway, but good practice as it doesn’t always

```
if world_size > 1 and dist.is_available():
  dist.destroy_process_group()
```



© 2025 Hewlett Packard Enterprise Development LP

Create a Distributed Data Sampler for training data

Use a distributed sampler when loading data via DDP

```
# Turn off shuffling when we do distributed sampling
if(world_size > 1):
  shuffle = False
else:
  shuffle = True

if use_cuda:
  cuda_kwargs = {'num_workers': 1,
                 'pin_memory': True,
                 'shuffle': shuffle}
train_kwargs.update(cuda_kwargs)
test_kwargs.update(cuda_kwargs)
...

# Create distributed sampler for training data if we are distributed
train_sampler = torch.utils.data.distributed.DistributedSampler(
dataset1, num_replicas=world_size, rank=rank) if world_size > 1 else None

# Add it to the training data loader
train_loader = torch.utils.data.DataLoader(
dataset1, sampler=train_sampler, **train_kwargs)
```



© 2025 Hewlett Packard Enterprise Development LP

Parallelize our training model with DDP

Wrap the existing model with DDP

```
from torch.nn.parallel import DistributedDataParallel as DDP
```

```
if dist.is_available() and world_size > 1:
  if use_cuda:
    model = DDP(model, device_ids=[local_rank])
  else:
    # device_ids not needed for CPU/MPS
    model = DDP(model)
```



© 2025 Hewlett Packard Enterprise Development LP

Adjust logging for distributed computing

Only log on **rank 0**

```
def train(args, model, device, train_loader, optimizer, epoch, rank):
…
  if batch_idx % args.log_interval == 0 and rank == 0:
    print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
          epoch, batch_idx * len(data), len(train_loader.dataset),
          100. * batch_idx / len(train_loader), loss.item()))
  if args.dry_run:
    break
```



© 2025 Hewlett Packard Enterprise Development LP

Saving model state with DDP

If we are using a DDP wrapper, we need to pull the model state out of the wrapper

```
if args.save_model and rank == 0:
  try:
    model_state = model.module.state_dict() if world_size > 1 else model.state_dict()
    torch.save(model_state, "mnist_cnn.pt")
  except Exception as e:
    print(f"Error saving model: {e}")
```



© 2025 Hewlett Packard Enterprise Development LP

Optional: Only test on rank 0

No need to run the test training on every rank for this example

```
# Test only on rank 0 when distributed
if(rank==0):
  try:
    dataset2 = datasets.MNIST('../data', train=False, transform=transform)
  except Exception as e:
    print(f"Error loading test dataset: {e}")
    exit(1)
…

if(rank==0):
  test_loader = torch.utils.data.DataLoader(dataset2, **test_kwargs)


…
if(rank == 0):
  test(model, device, test_loader)
```



© 2025 Hewlett Packard Enterprise Development LP

A SLURM Batch script for distributed training

```
#!/bin/bash
#SBATCH --job-name=Torch_Distributed
#SBATCH --nodes=2
#SBATCH --gpus=8
#SBATCH --time=00:10:00
#SBATCH --ntasks-per-node=4

module load brics/nccl brics/aws-ofi-nccl
module load cray-python
cd $SLURM_SUBMIT_DIR
source ./.venv/bin/activate

export MASTER_ADDR=$(scontrol show hostname $SLURM_NODELIST | head -n 1) # e.g. nid001038
export MASTER_PORT=29600

# Run the function with srun
srun -N 2 \
--gpus=8 \
--ntasks-per-node=4 \
bash -c 'export WORLD_SIZE=$SLURM_NTASKS; export RANK=$PMI_RANK; export LOCAL_RANK=$SLURM_LOCALID; $NUMA_CMD python3 main-ddp.py'
```



© 2025 Hewlett Packard Enterprise Development LP

Other things you can try

- Pinning / not pinning
- PyTorch’s built in profiling
- Add snapshotting & restore from snapshot to your code
- Try launching with torchrun



© 2025 Hewlett Packard Enterprise Development LP